

GUI 智能体项目理解与实施计划

一、项目总体理解

本项目拟开发一个基于多模态大模型的桌面 GUI 智能体原型，使系统能够接收自然语言指令，理解当前屏幕内容，生成可执行的任务步骤，并通过鼠标、键盘等工具完成界面操作，最终反馈执行结果。

项目核心流程为：

用户指令 → 屏幕感知 → 任务理解与规划 → 动作生成与执行 → 结果检查与反馈

项目的重点不只是调用大模型，而是建立一个可验证的桌面任务执行闭环，具体包括以下四类能力：

- 1. 屏幕感知：**获取屏幕截图，识别文字、图像区域和可交互元素，并建立视觉结果与屏幕坐标之间的对应关系。
- 2. 任务规划：**把用户目标拆解为可执行步骤，根据当前界面状态决定下一步操作。
- 3. 动作执行：**完成点击、输入、滚动、拖拽等基础操作，并保留执行日志。
- 4. 结果反馈：**检查操作后的界面变化，判断步骤是否成功，并输出任务结果或失败原因。

项目最终应形成一个能够运行、测试和复现的原型，而不是只完成模型接口或单个自动化脚本。本次项目将优先完成前四周的任务，即技术调研与环境搭建、桌面感知与控制模块、公开 GUI 数据处理与基础 Agent、端到端 GUI 任务执行系统集成。

大纲第5至第8周涉及的 LoRA 微调、高级功能、系统全面评估和求职作品集整理，作为提前完成主线任务后的扩展方向。

二、开发平台与技术路线

2.1 主开发平台

项目以 **Windows 11** 为主要开发、运行和验收平台，使用本地 NVIDIA RTX 5070 Ti 进行模型推理与小规模实验。时间允许的情况下会对macos和linux平台进行兼容性测试和优化。

2.2 初步技术路线

模块	初步方案	使用原则
开发环境	Python 3.10+、PyTorch 2.2+、CUDA、Git、GitHub	固化依赖版本和配置步骤，保证环境可复现
屏幕截图	mss、PyAutoGUI	mss 负责高性能截图，PyAutoGUI 用于基础截图与自动化能力验证
图像与文字识别	OpenCV、PaddleOCR，EasyOCR 作为备选	OpenCV 负责预处理、模板匹配和边界框绘制；OCR 负责文字及文字坐标识别
鼠标键盘控制	PyAutoGUI、pynput	支持点击、输入、滚动和拖拽，并增加中止机制与执行日志
多模态模型	参考 Qwen-VL、GLM-4V、Llama 3.2 Vision 等候选方案	第一周评估部署可行性，第三周收敛为一套主方案
Agent 组织	LangChain、LlamaIndex 或轻量自研流程	根据调试透明度和集成成本择一，不同时堆叠多个框架
模型推理	Transformers；满足兼容条件时评估 vLLM	优先实现稳定的最小推理和统一模型调用接口
交互方式	命令行界面	四周内不额外开发桌面前端，避免偏离核心算法任务

项目测试仅在授权的 Windows 本地环境、测试账号或模拟应用中进行。“发送消息”等可能产生外部影响的任务使用测试对象完成，不向真实联系人发送消息，也不操作个人敏感数据。

三、四周任务范围与交付安排

实际周次	日期	对应大纲任务	本周核心目标	周一提交时间
第1周	8月4日—8月9日	大纲第1周	技术调研与 Windows 开发环境搭建	8月10日
第2周	8月10日—8月16日	大纲第2周	桌面感知与控制核心模块开发	8月17日
第3周	8月17日—8月23日	大纲第3周	公开 GUI 数据处理与基础 Agent 搭建	8月24日
第4周	8月24日—8月30日	大纲第4周	端到端 GUI 任务执行系统集成	8月31日

每周一提交上一周期的：

- 本周代码及对应 Git commit；
- 相关的交付物；
- 一篇周报；

四、第一周计划：技术调研与基础环境搭建

4.1 工作内容

1. 阅读项目大纲，明确四周项目边界，并调研 UI-TARS、Claude Computer Use、ScreenAgent 等代表性方案。
2. 对比各方案的屏幕感知、任务规划、动作空间、结果反馈和安全机制，梳理 GUI 智能体的核心架构与关键挑战。
3. 在 Windows 11 上配置 Python、PyTorch、CUDA、Git 等基础开发环境，确认 RTX 5070 Ti 可被深度学习框架正确识别。
4. 安装并验证 mss、PyAutoGUI、OpenCV、PaddleOCR、pynput 等基础工具，运行截图、OCR 和桌面控制的最小示例。
5. 评估候选多模态模型的显存需求、加载方式和调用接口，确定主方案与备用方案。
6. 确定项目技术路线、模块边界、基础数据结构和第二周接口约定。
7. 整理环境复现步骤、验证结果、问题风险和下一周计划，完成第一周材料与周报。

4.2 第一周交付物

- 《GUI 智能体技术调研报告》；
- 系统初步架构图；
- 《Windows 开发环境配置文档》；
- 第一周周报；
- 第一周代码与文档的 Git 提交记录。

4.3 第一周验收标准

- 调研报告能够说明代表性方案的核心架构、技术差异、适用边界和主要风险；
- Windows 11 开发环境可以正确识别 RTX 5070 Ti，Python、PyTorch 和 CUDA 的关键版本信息记录完整；
- 截图、OCR 和基础鼠标键盘控制的最小示例可以在受控环境中运行；
- 环境配置文档包含必要的安装命令、依赖版本和常见问题，可用于重新搭建环境；
- 失败项有对应的错误日志、原因分析和替代方案；
- 第一周代码、文档和周报内容相互一致，且不包含密钥、个人数据或无关大文件。

五、第二周计划：桌面感知与控制模块

5.1 工作内容

1. 实现 Windows 屏幕实时截图，验证常用分辨率和缩放比例。
2. 集成 OCR，输出文字内容、置信度和屏幕坐标。
3. 使用 OpenCV 完成图像预处理、模板匹配、边界框绘制和结果可视化。
4. 开发鼠标键盘控制模块，支持点击、输入、滚动和拖拽。
5. 统一感知与控制模块的输入输出格式。
6. 增加紧急中止、操作间隔、坐标越界检查和日志记录。
7. 编写单元测试与受控环境下的集成测试。

5.2 第二周交付物

- 屏幕截图模块；
- OCR 与坐标定位模块；
- 图像处理和边界框可视化模块；
- 鼠标键盘控制模块；
- 单元测试与测试样例；
- 《桌面感知与控制模块测试报告》；
- 第二周周报。

5.3 第二周验收标准

- 能在 Windows 主开发机上完成截图并正确记录屏幕尺寸和缩放信息；
- 能输出测试界面中的文字、置信度及边界框坐标；
- 能在受控测试窗口中完成点击、输入、滚动和拖拽；
- 控制模块具备手动中止能力，不对真实业务软件执行不可逆操作；
- 测试报告记录成功案例、失败案例和已知限制。

六、第三周计划：数据处理与基础 Agent

6.1 工作内容

1. 调研并获取 ScreenAgent、Mind2Web、WebArena 等公开 GUI 数据、任务轨迹或评测环境。
2. 梳理不同来源的数据结构、许可范围和适用场景，选择与桌面任务最相关的部分。
3. 编写数据预处理脚本，统一指令、截图、动作和结果字段。
4. 搭建基础 Agent，定义观察、规划、动作和反馈的数据结构。
5. 实现简单任务拆解与逐步执行逻辑。
6. 开发统一模型调用接口，优先支持一套本地多模态模型，并为必要的 API 调用保留适配层。
7. 将第三周模块与第二周感知、控制接口进行最小联调。

6.2 第三周交付物

- 数据预处理脚本及少量处理后样例；
- 基础 Agent 框架代码；

- 任务拆解与规划模块；
- 统一模型调用接口；
- 第三周周报。

6.3 第三周验收标准

- 数据脚本可重复运行，并能输出统一格式的样例；
- Agent 能接收用户任务和屏幕观察结果，输出结构化执行计划；
- 模型调用失败时能返回明确错误，不直接触发无约束桌面操作；
- 感知、规划和控制模块之间的接口定义清晰且有样例。

七、第四周计划：端到端系统集成

7.1 工作内容

1. 集成用户指令、屏幕感知、任务规划、动作执行和结果反馈模块。
2. 开发简单命令行交互界面。
3. 实现步骤级状态记录、超时限制和失败退出。
4. 在受控 Windows 环境中测试五个基础任务。
5. 记录每项任务的执行步骤、成功与否、耗时和失败原因。
6. 修复影响主闭环的问题，整理运行说明和复现步骤。

7.2 第四周交付物

- 端到端 GUI 智能体原型 v1.0；
- 命令行交互入口与运行脚本；
- 五项基础任务的测试用例和执行日志；
- 《基础任务测试报告》；
- 第四周周报与最终交付清单。

7.3 第四周验收标准

- 新环境可根据 README 完成安装并启动系统；
- 系统能形成“指令—感知—规划—执行—反馈”的完整闭环；
- 五项基础任务均有可复现测试过程，不隐藏失败案例；
- 测试报告至少统计任务成功情况、执行时间和主要失败原因；

八、周报模板

周报统一采用以下结构：

1. **本周目标**：本周计划解决的问题和预期交付物。
2. **完成情况**：逐项说明已完成、部分完成和未完成内容。
3. **验证结果**：测试环境、测试方法、关键结果及材料链接。
4. **问题与分析**：当前问题、失败原因和影响范围。
5. **风险与调整**：进度、算力、数据、模型或兼容性风险，以及应对措施。
6. **下周计划**：下一周期任务、优先级和预期交付物。

九、扩展目标

在原大纲第1至第4周任务全部完成且主闭环稳定后，再按剩余时间选择以下扩展项，不影响必交任务验收：

- 1. 使用 PEFT 完成小规模 LoRA/QLoRA 可行性实验；
- 2. 增加失败检测、有限重试和任务状态监控；
- 3. 扩展测试任务并统计成功率、平均执行时间和错误率；
- 4. 优化 UI 识别速度和坐标偏移；
- 5. 完善技术报告、演示视频和项目作品集材料。